ASSIGNMENT

Custom Reinforcement Learning Agent Implementation using Farama Gymnasium

Reinforcement Learning / AI Assignment

Environment: CliffWalking-v1 | Algorithm: Q-Learning

Name: Azhar Mehmood

Reg. No: FA23-BAI-039

Submission: GitHub Repository Link + Technical Report (PDF)

Due Date: 20/10/2026

Technical Report

Azhar Mehmood (FA23-BAI-039) | CliffWalking-v1 | Q-Learning from scratch

1. Problem Description

For this assignment I picked **CliffWalking-v1** from Gymnasium. It is a small 4x12 grid (48 states), so I could implement the algorithm myself without needing neural networks. The agent starts at the bottom-left cell and has to reach the goal at the bottom-right. There is a cliff along the bottom row. If the agent steps into the cliff, it gets a big penalty and goes back to the start. Actions are simple: **0 = Up, 1 = Right, 2 = Down, 3 = Left**.

Reward setup is also straightforward. Every step gives **-1**. Falling into the cliff gives **-100**. Reaching the goal ends the episode. The best safe path is 13 steps, so the best score is **-13**. If the agent takes a longer upper path, the score gets worse (more negative). That tradeoff is why this environment is useful for testing an RL agent.

2. Methodology

2.1 Algorithm

I implemented **Q-Learning** with an epsilon-greedy policy. The idea is to keep a table Q(s, a) for every state-action pair. Most of the time the agent picks the action with the highest Q value, but with probability epsilon it picks a random action so it can explore. After each step, the table is updated with:

$$Q(s,a) = Q(s,a) + \alpha * [\, r + \gamma * \max Q(s',a') - Q(s,a) \,]$$

Here alpha is the learning rate and gamma is the discount factor. Epsilon starts at 1.0 and slowly decays after every episode down to 0.01. I used the max over next actions in the update, which is the normal off-policy Q-Learning rule.

2.2 Code Structure

I split the code into three files so it stays easy to read. **agent.py** has the QLearningAgent class (choose action, update Q, decay epsilon, save/load). **utils.py** has training, evaluation, plots, and GIF rendering. **main.py** runs the full pipeline from the terminal. The environment is created with *gymnasium.make("CliffWalking-v1")*. After training, I save the Q-table as *models/q_table.npy* and a short demo clip as *results/agent_demo.gif*.

3. Experimental Setup and Hyperparameters

I trained in Python using Gymnasium. The table below shows the final settings I kept for the saved model and GIF. For the sensitivity test, I trained again for 3000 episodes and changed one setting at a time (same seed).

Hyperparameter	Symbol	Value	Why I chose it
Learning rate	alpha	0.5	0.1 was slow; 0.9 was less stable
Discount factor	gamma	0.99	Helps the agent care about reaching goal
Start epsilon	eps0	1.0	Start fully random
Min epsilon	eps_min	0.01	Keep a little exploration
Epsilon decay	eps_decay	0.995	Decay once per episode
Training episodes	N	5000	Main training run
Max steps	T	200	Stop very long episodes
Eval episodes	-	100	Test greedy policy
Seed	-	42	Same seed for fair comparison

4. Results and Discussion

After 5000 episodes, I tested the greedy policy for 100 episodes. The average return was **-13.0** with **0** variance, and success rate was **100%**. That means the agent learned the optimal 13-step path. Looking at Figure 1, reward improves a lot in the first 1000-2000 episodes, then becomes more stable as epsilon gets small. Figure 2 shows the learned values on the grid. Cells near the good path have higher values, and cells near the cliff look worse, which makes sense.

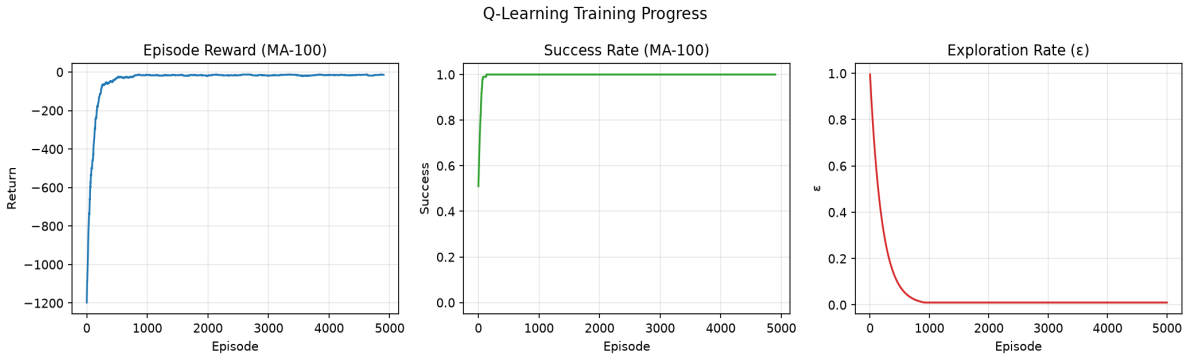


Figure 1. Reward, success rate, and epsilon during training.

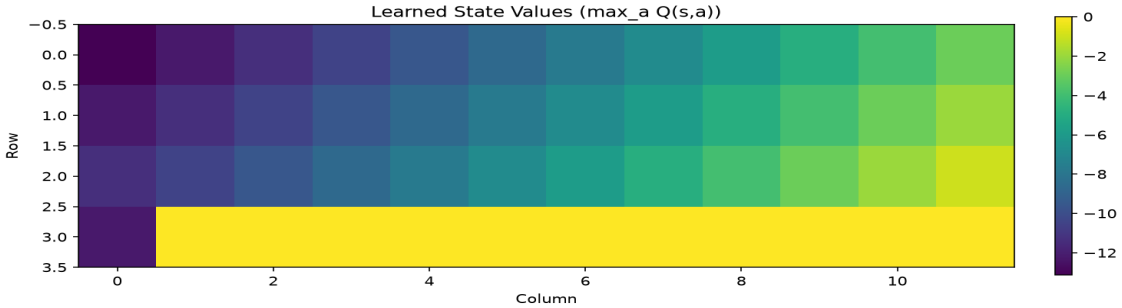


Figure 2. Max Q-value for each cell on the 4x12 map.

For hyperparameter checks (Figure 3), alpha = 0.5 worked better than alpha = 0.1 in the same episode budget. Gamma = 0.99 was fine for this task. When I decayed epsilon too slowly (0.999), the agent kept exploring for longer, so the late training return stayed worse. Overall, learning rate and epsilon decay affected the results the most in my runs.

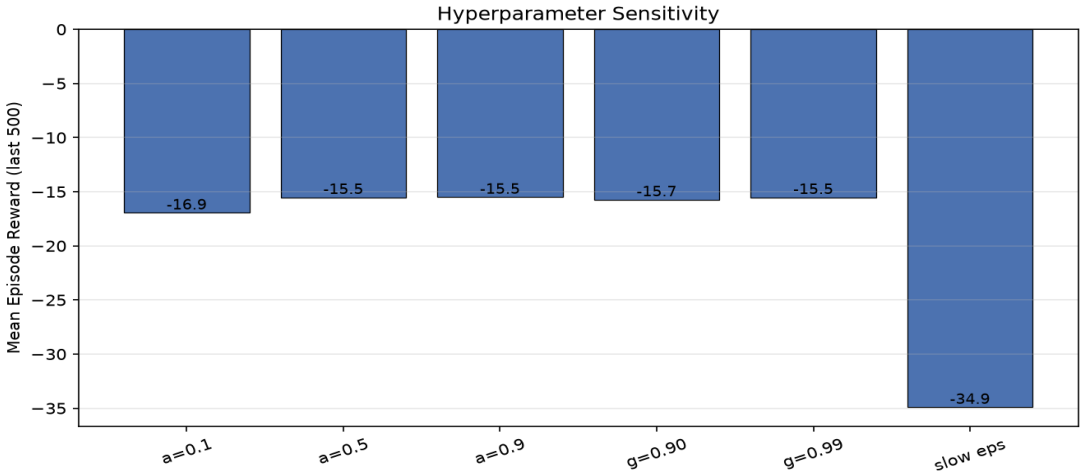


Figure 3. Average return in the last 500 episodes for different settings.

4.1 Problems I Noticed

Early in training the agent falls into the cliff a lot because epsilon is high. That looks bad in the reward plot, but it is normal while it is still exploring. Sometimes the agent can also learn a safer but longer path on the upper rows, which gives a worse score than -13. Another limit is that this is a table method, so it works for CliffWalking, but it would not directly work for continuous games like CartPole unless I switch to something like DQN. The demo GIF shows the final greedy policy walking from start to goal without falling.

5. Conclusion

In this assignment I built a Q-Learning agent from scratch on CliffWalking-v1, trained it, plotted the learning curves, saved the Q-table, and recorded a short GIF of the final policy. The agent reached the optimal score (-13) in evaluation. Changing alpha, gamma, and epsilon decay clearly changed how fast and how well it learned. If I continue this work, I would try SARSA (usually safer near the cliff) or Double Q-Learning next.

GitHub: https://github.com/azhar-fullstack/RL-Assignment-CliffWalking-QLearning
Main files: main.py, agent.py, utils.py, notebooks/training.ipynb, models/q_table.npy, results/agent_demo.gif